B.M.S COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU

AAT Report on

Design and verification of Digital circuits using Cadence Tool

Submitted in partial fulfilment of AAT
for the course on
Advanced Digital Logic Design

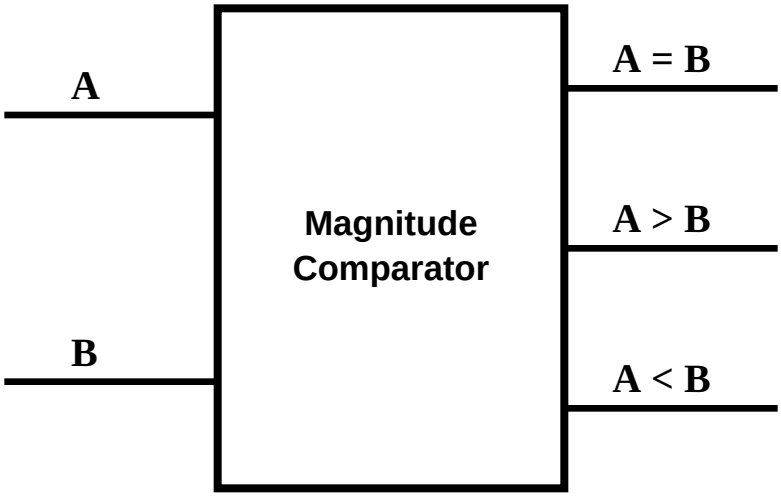
By,
VISHAL B SANTI
1BF24EC186

To
Prof. Ajaykumar Devarapalli
Assistant Professor

Department of electronics and communication

B.M.S College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
2025-2026

COMPARATOR



Introduction to Comparator

- A comparator is a combinational logic circuit used to compare two binary numbers.
- It takes two binary inputs, A and B.
- It determines whether A is greater than, less than, or equal to B.
- A 4-bit comparator produces three outputs: A > B, A < B, and A = B.
- Comparators are used in ALUs, processors, digital systems, and control circuits.
- The design is parameterised (N = 4) and written using continuous assignments with the relational operators >, < and ==.

Code:

```
// comparator RTL
module comparator #(parameter N = 4)
(
    input [N-1:0] A, B,
    output A_gt_B,
    output A_lt_B,
    output A_eq_B
);

assign A_gt_B = (A > B);
assign A_lt_B = (A < B);
assign A_eq_B = (A == B);

endmodule
```

Testbench:

```
// test bench
module tb_comparator;

parameter N = 4;

reg [N-1:0] A, B;
wire A_gt_B, A_lt_B, A_eq_B;

comparator #(N) uut (
    .A(A),
    .B(B),
    .A_gt_B(A_gt_B),
    .A_lt_B(A_lt_B),
    .A_eq_B(A_eq_B)
);

initial begin
    $dumpfile("comparator.vcd");
    $dumpvars(0, tb_comparator);

    $monitor("Time=%0t A=%b B=%b GT=%b LT=%b EQ=%b",
             $time, A, B, A_gt_B, A_lt_B, A_eq_B);

    A = 4'b0101; B = 4'b0011;
    #10;

    A = 4'b0010; B = 4'b0110;
    #10;

    A = 4'b1010; B = 4'b1010;
    #10;

    A = 4'b1111; B = 4'b0001;
    #10;

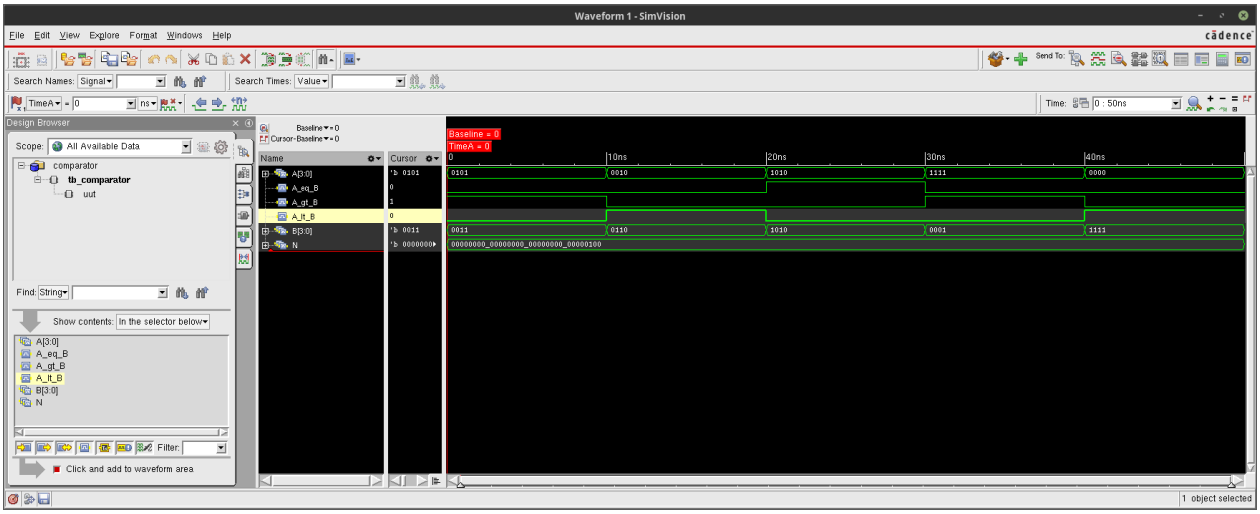
    A = 4'b0000; B = 4'b1111;
    #10;

    $finish;

end

endmodule
```

Output:



Inference

- 0ns-10ns: A = 0101 (5) > B = 0011 (3), so A_gt_B = 1.
- 10ns-20ns: A = 0010 (2) < B = 0110 (6), so A_lt_B = 1.
- 20ns-30ns: A = 1010 = B = 1010, so A_eq_B = 1.
- 30ns-40ns: A = 1111 (15) > B = 0001 (1), so A_gt_B = 1.
- 40ns-50ns: A = 0000 (0) < B = 1111 (15), so A_lt_B = 1.
- Exactly one output is high at any instant, confirming correct comparator behaviour.

DIVIDE-BY-N COUNTER

Introduction to Divide-by-N Counter

- A divide-by-N counter is a sequential circuit that counts clock pulses from 0 to N-1 and then rolls over.
- It has two inputs: clk (clock) and rst (asynchronous, active-high reset).
- It has two outputs: count (the counter value, 4 bits wide for N = 10) and out (the divided clock).
- When rst = 1, count and out are cleared to 0.
- When count reaches N-1, it returns to 0 and out toggles, so out changes state every N clock cycles.
- It is used for clock division, timers and frequency synthesis. Here N = 10, so count runs 0 to 9.

Code:

tb_counter.v | divide_n_counter.v

```
// divide by n counter RTL
module divide_n_counter #(
    parameter N = 10
)(
    input clk,
    input rst,
    output reg [$clog2(N)-1:0] count,
    output reg out
);

always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        count <= 0;
        out <= 0;
    end
    else
    begin
        if (count == N-1)
        begin
            count <= 0;
            out <= ~out;
        end
        else
        begin
            count <= count + 1;
        end
    end
end

endmodule

```

Testbench:

```
// test bench
module tb_counter;

reg clk, rst;
wire [3:0] count;
wire out;

divide_n_counter #(.N(10)) uut (
    .clk(clk),
    .rst(rst),
    .count(count),
    .out(out)
);

always #5 clk = ~clk;

initial
begin
    // Generate waveform dump
    $dumpfile("divide_n_counter.vcd");
    $dumpvars(0, tb_counter);

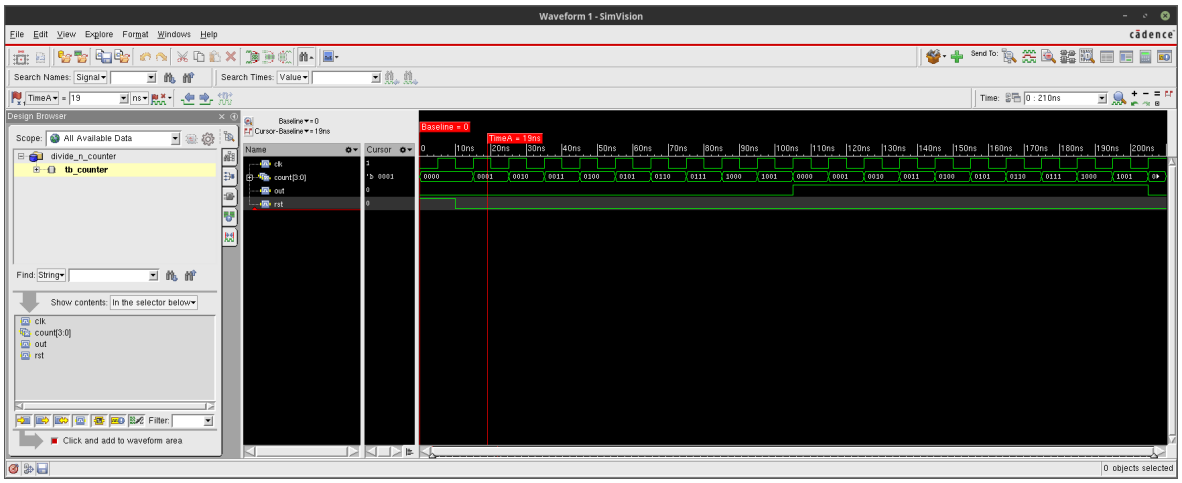
    clk = 0;
    rst = 1;

    #10 rst = 0;

    #200 $finish;
end

endmodule
```

Output:



Inference

- count increments 0000 to 1001 (0 to 9) on each rising clock edge and then wraps to 0000.
- out toggles each time count wraps, i.e. every 10 clock cycles (100ns here).